



UNIVERSITY OF
TORONTO

Applied Deep Learning

Lecture 3: *Multiclass Classification and Backpropagation*

Ali Bereyhi

ECE Department, University of Toronto

Fall 2026



UNIVERSITY OF
TORONTO

Lecture Overview

Quick Recap: *ML Components*

Any ML problem has *three components*:

1. *Dataset* which is the collection of samples

$$\boxed{\text{supervised learning}} \quad \mathbb{D} = \{(\mathbf{x}_i, y_i) : i = 1, \dots, I\}$$

2. *Model* that describes the relation between the *input* and *label*

$\mathbf{h}$: Hyperparameters

$$z = f_{\mathbf{h}}(\mathbf{x} | \mathbf{w})$$

$\mathbf{w}$: Learnable Parameters

3. *Loss* quantifies the difference between the *model's output* and *true label*

$$\mathcal{L}(z_i, y_i)$$

Recap: *Training*

The *training* is performed by minimizing the empirical risk

$$\begin{aligned}\mathbf{w}^{\star} &= \underset{\mathbf{w}}{\operatorname{argmin}} \hat{R}(\mathbf{w}) \\ &= \underset{\mathbf{w}}{\operatorname{argmin}} \frac{1}{I} \sum_{i=1}^I \mathcal{L}(f_{\mathbf{h}}(\mathbf{x}_i | \mathbf{w}), y_i)\end{aligned}\quad (\text{Training})$$

Training Algorithm

We can train using the gradient descent algorithm



Lecture 3. *Multiclass Classification and Backpropagation*

Today, we are going to learn

1. Multiclass classification via practical models
2. Representing NNs on a computation graph
 - *This enables us to develop **backpropagation algorithm***



UNIVERSITY OF
TORONTO

Multiclass Classification

Binary Classification on MNIST

In MNIST, we have a set *hand-written digits*, something like this



We intend to train an MLP that classifies hand-written digit as

digit "2" or anything else

MNIST Classification: *ML Components*

Data. We get a *batch* of B images to train, i.e.,

$$\mathbb{D} = \{(\mathbf{x}_b, v_b) \text{ for } b = 1, \dots, B\}$$

- $\mathbf{x}_b \in \mathbb{R}^{784}$ is the *pixel vector* of image b
- $v_b \in \{0, 1\}$ is a binary label indicating *whether it is “2” or not*

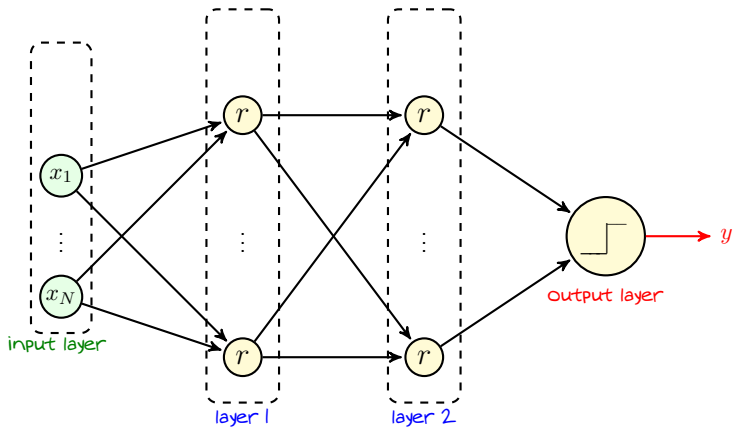
Model. We consider an MLP with *two hidden layers*

- All *hidden neurons* are activated by **ReLU**

$$r(x) = \text{ReLU}(x)$$

- **Output layer** has a *single perceptron*

MNIST Classification: *Model*



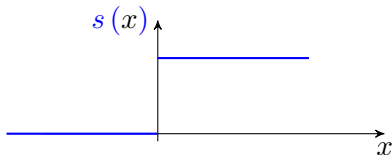
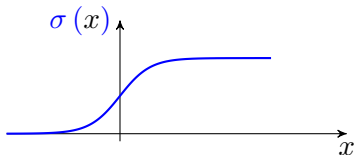
MNIST Classification: *Differentiable Activation*

- + The activation in *perceptron* is *not differentiable*!
- We replace it with a *differentiable* approximation of *step function*

We already have seen the *sigmoid* function

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

which looks pretty close to step function



MNIST Classification: *Differentiable Loss*

- + But, the *output* of the network is now *not binary*!
- We now interpret the *output* as *probability*

y is the *probability* of the *label being 1*

- + OK! But how can we define the *loss* now?
- We could look at the *true label* from the same point of view

Say $v \in \{0, 1\}$ is *true label*:

- if $v = 1 \rightsquigarrow$ *true label is 1* with *probability 1*
- if $v = 0 \rightsquigarrow$ *true label is 1* with *probability 0*

So, we could say

true label is 1 with *probability v*

MNIST Classification: *MSE Loss*

*One may suggest that we use the **squared error**, i.e.,*

$$\mathcal{L}(y, v) = (y - v)^2$$

in this case the empirical risk is called

Mean Squared Error (MSE)

This loss is **differentiable**

$$\dot{\mathcal{L}}(y, v) = 2(y - v)$$

and proportional to the distance between y_3 and v

*It's a **good** choice but **not best***

MNIST Classification: *Cross-Entropy*

A better choice is to determine the *cross-entropy loss*

$$\begin{aligned}\mathcal{L}(y, v) &= \text{CE}(y, v) = -v \log y - (1 - v) \log (1 - y) \\ &= \begin{cases} \log \frac{1}{y} & v = 1 \\ \log \frac{1}{1-y} & v = 0 \end{cases}\end{aligned}$$

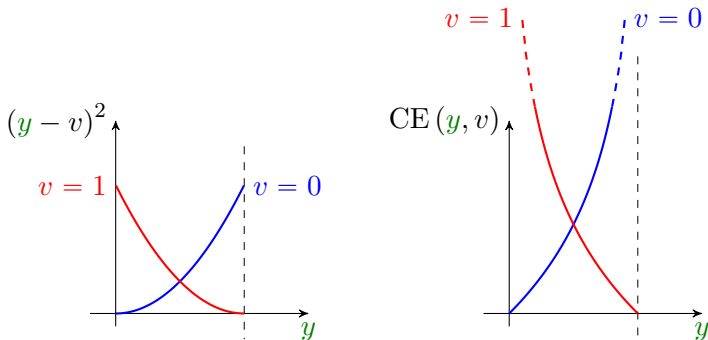
This loss is again *differentiable*

$$\dot{\mathcal{L}}(y, v) = \dot{\text{CE}}(y, v) = -\frac{v}{y} + \frac{1 - v}{1 - y}$$

Note. The logarithm is usually in natural base, i.e., $\log x = \ln x$

Cross-Entropy

- + But why *cross-entropy* is a *better loss*?
- It pushes *y* more towards the edges of interval $[0, 1]$



Cross entropy returns *much higher loss* when *y* is different from *v*

MNIST Classification: *Inference*

- + How do we *infer the label* from the *output* after training?!
- Just follow the *interpretation*

y gives the *probability* of the *image* being *hand-written “2”*

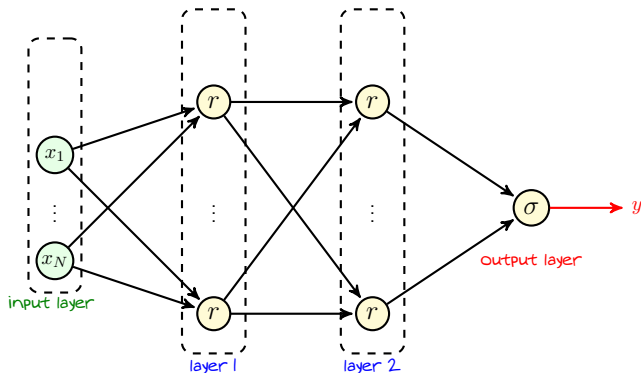
↳ $(1 - y_3)$ gives the *probability* of image being *any other digit*

So, we select the outcome with *higher chance*, i.e.,

- if $y \geq 0.5 \rightsquigarrow$ label the *image* as *digit “2”*
- if $y < 0.5 \rightsquigarrow$ label the *image* as *other digits*

- + Can't we classify more classes? Like hand-written “0”, “1”, ..., “9”?
- Sure! We can!

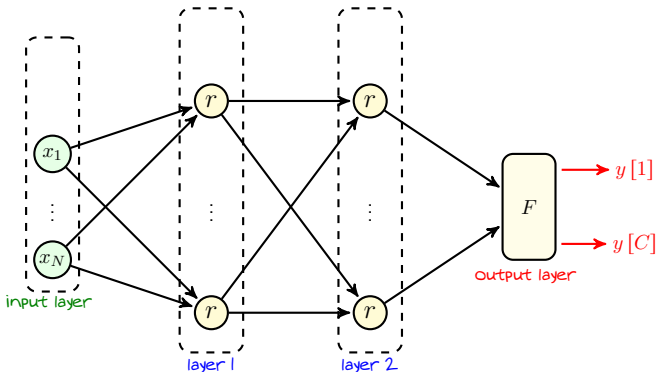
Multiclass MNIST Classification: *Model*



We interpreted y as the *probability of label being 1*

probability of label being 0 is hence $1 - y$

Multiclass MNIST Classification: *Model*



With C classes, we need a module that computes *probabilities* of all C classes
 this module is a *neuron* with *vector output*

Multiclass Classification: *Softmax*

Softmax Function

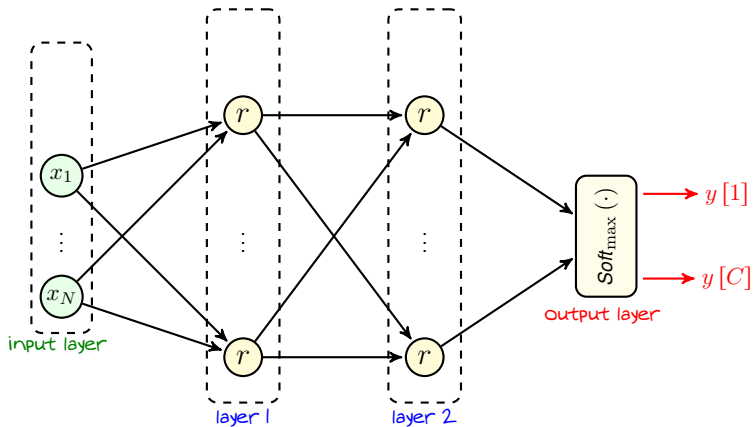
For $\mathbf{z} \in \mathbb{R}^C$, softmax function returns $\text{Soft}_{\max}(\mathbf{z}) = \mathbf{y} \in \mathbb{R}^C$ whose entry c is

$$y[c] = \frac{e^{z[c]}}{\sum_{j=1}^C e^{z[j]}}$$

Softmax returns a probability distribution on the set of classes

$$\sum_{c=1}^C y[c] = \sum_{c=1}^C \frac{e^{z[c]}}{\sum_{j=1}^C e^{z[j]}} = \frac{\sum_{c=1}^C e^{z[c]}}{\sum_{j=1}^C e^{z[j]}} = 1$$

Multiclass Classification: *Softmax*



Multiclass Classification: Loss

- + How can we define the loss now?
- Again we need to convert *true labels* to *probabilities*

Let's say we have C classes: the *vector* of *probabilities* contains C entries

$$\mathbf{p} = \begin{bmatrix} p_1 \\ \vdots \\ p_C \end{bmatrix} = \begin{bmatrix} \Pr \{ \text{image belongs to class } 1 \} \\ \vdots \\ \Pr \{ \text{image belongs to class } C \} \end{bmatrix}$$

If we know that the *image* b belongs to class v_b , we could say that

$$\mathbf{p} \text{ of image } b = \begin{bmatrix} p_1 \\ \vdots \\ p_{v_b} \\ \vdots \\ p_C \end{bmatrix} = \begin{bmatrix} \Pr \{ \text{image } b \text{ belongs to class } 1 \} = 0 \\ \vdots \\ \Pr \{ \text{image } b \text{ belongs to class } v_b \} = 1 \\ \vdots \\ \Pr \{ \text{image } b \text{ belongs to class } C \} = 0 \end{bmatrix} = \begin{bmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix}$$

Multiclass Classification: Loss

So, we could say that *label v is corresponding to a vector of size C whose entry v is 1 and the remaining entries are all 0*: this vector is called a *one-hot vector*

One-hot Vector

The one-hot vector $\mathbf{1}_v \in \{0, 1\}^C$ is a C -dimensional vector whose entry v is 1 and all remaining entries are 0

For instance: say $C = 3$; then, we have

$$\mathbf{1}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \quad \mathbf{1}_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \quad \mathbf{1}_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

Moral of Story

*We can interpret *true label* v as a probability vector $\mathbf{1}_v$*

Multiclass Classification: Loss

Cross entropy can also be extended as follows

$$\begin{aligned}\mathcal{L}(\mathbf{y}, \mathbf{1}_v) &= \text{CE}(\mathbf{y}, \mathbf{1}_v) = - \sum_{c=1}^C \mathbb{1}\{c = v\} \log y(c) \\ &= -\log y[v]\end{aligned}$$

The gradient of this loss is

$$\nabla \mathcal{L}(\mathbf{y}, \mathbf{1}_v) = \nabla \text{CE}(\mathbf{y}, \mathbf{1}_v) = \begin{bmatrix} 0 \\ \vdots \\ -1/y[v] \\ \vdots \\ 0 \end{bmatrix} = -\frac{1}{y[v]} \mathbf{1}_v$$



UNIVERSITY OF
TORONTO

Computation Graph and Backpropagation

MLPs: *Forward Pass*

Let us now see *how a given data-point propagates through the FNN*: we want to write the **outputs** $y_1, \dots, y_M$ when **inputs** $x_1, \dots, x_N$ are given

this is called *forward propagation* through the network

or simply

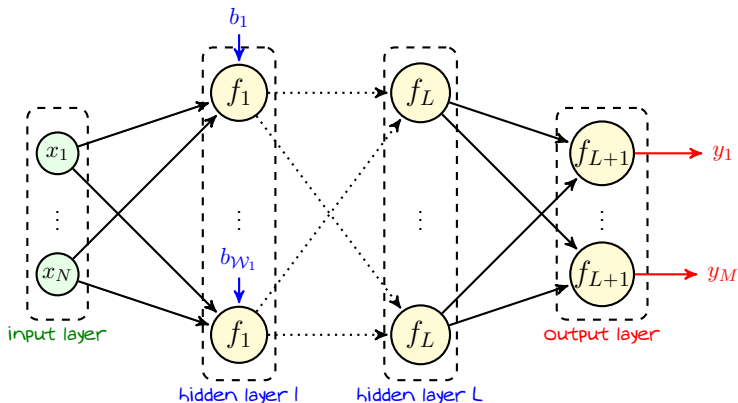
the *forward pass*

which tracks values *passed* through the NN from the **input** to **output layer**

Fully-Connected FNNs: *Few Definitions*

We can get rid of biases by defining a new constant node in each layer

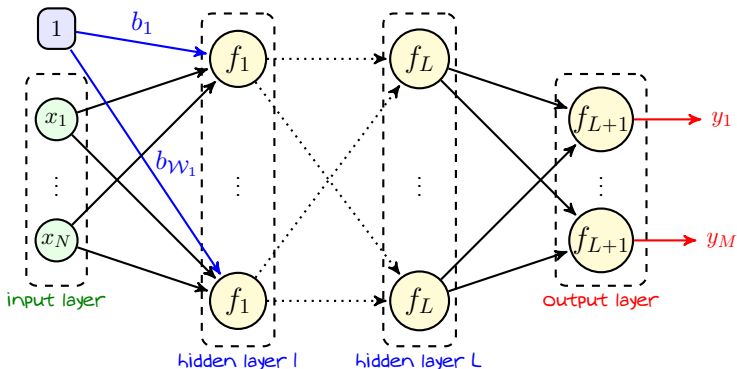
Let's look at the first layer: we have $\mathcal{W}_1$ *neurons* and each has *a bias*



Fully-Connected FNNs: *Few Definitions*

We can get rid of biases by defining a new constant node in each layer

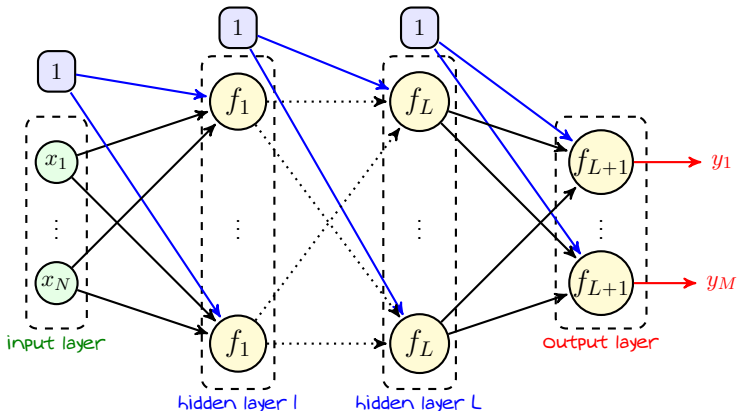
We introduce a constant input and let these biases being the weights of its links



Fully-Connected FNNs: *Few Definitions*

We can get rid of biases by defining a new constant node in each layer

We do the same in all layers: now neurons have no biases



MLPs: Forward Pass

We have three types of nodes in MLPs

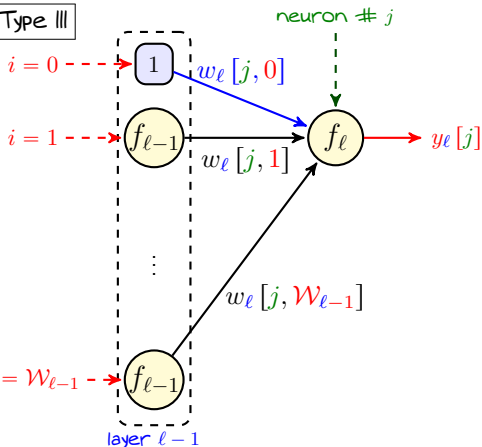
Type II



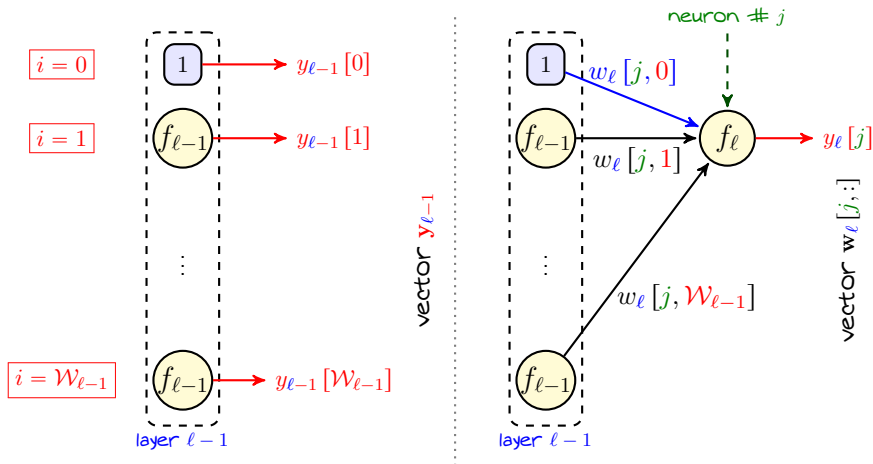
Type I



Type III



Fully-Connected FNNs: *Forward Pass*



MLPs: *Forward Pass*

We can represent $z_j [\ell]$ more compactly via vectorized notation

$$\begin{aligned} z_{\ell} [j] &= \sum_{i=0}^{\mathcal{W}_{\ell-1}} w_{\ell} [j, i] y_{\ell-1} [i] \\ &= \underbrace{[w_{\ell} [j, 0] \quad w_{\ell} [j, 1] \quad \dots \quad w_{\ell} [j, \mathcal{W}_{\ell-1}]]}_{\mathbf{w}_{\ell} [j, :]^T} \underbrace{\begin{bmatrix} y_{\ell-1} [0] \\ y_{\ell-1} [1] \\ \vdots \\ y_{\ell-1} [\mathcal{W}_{\ell-1}] \end{bmatrix}}_{\mathbf{y}_{\ell-1}} \\ &= \mathbf{w}_{\ell} [j, :]^T \mathbf{y}_{\ell-1} \end{aligned}$$

Fully-Connected FNNs: *Forward Pass*

We can further extend vectorized notation by defining

$$\mathbf{z}_\ell = \begin{bmatrix} z_\ell [1] \\ \vdots \\ z_\ell [\mathcal{W}_\ell] \end{bmatrix}$$

and thus writing $\mathbf{y}_\ell$ as

$$\mathbf{y}_\ell = f_\ell(\mathbf{z}_\ell)$$

where $f_\ell(\cdot)$ is applied entry-wise

and don't forget to add the dummy input 1, i.e.,

$$\mathbf{y}_\ell \leftarrow \begin{bmatrix} 1 \\ \mathbf{y}_\ell \end{bmatrix}$$

MLPs: *Forward Pass*

```
1: Initiate the output of the first layer as  $\mathbf{y}_0 = \mathbf{x}$ 
2: for  $\ell = 0, \dots, L$  do
3:   for  $j = 1, \dots, \mathcal{W}_{\ell+1}$  do
4:     Add  $\mathbf{y}_\ell[0] = 1$  and set  $z_{\ell+1}[j] = \mathbf{w}_{\ell+1}[j, :]^\top \mathbf{y}_\ell$       # affine function
5:   end for
6:   Compute  $\mathbf{y}_{\ell+1} = f_{\ell+1}(\mathbf{z}_{\ell+1})$                                 # activation
7: end for
8: for  $\ell = 1, \dots, L + 1$  do
9:   Return  $\mathbf{y}_\ell$  and  $\mathbf{z}_\ell$ 
10: end for
```

Fully-Connected FNNs: *Forward Pass*

We can present everything even more compactly: *let's write down $\mathbf{z}_\ell$*

$$\mathbf{z}_\ell = \begin{bmatrix} z_\ell[1] \\ \vdots \\ z_\ell[\mathcal{W}_\ell] \end{bmatrix} = \begin{bmatrix} \mathbf{w}_\ell[1, :]^\top \mathbf{y}_{\ell-1} \\ \vdots \\ \mathbf{w}_\ell[\mathcal{W}_\ell, :]^\top \mathbf{y}_{\ell-1} \end{bmatrix} = \begin{bmatrix} \mathbf{w}_\ell[1, :]^\top \\ \vdots \\ \mathbf{w}_\ell[\mathcal{W}_\ell, :]^\top \end{bmatrix} \mathbf{y}_{\ell-1}$$

Now, we can define the matrix $\mathbf{W}_\ell$ as

$$\mathbf{W}_\ell = \begin{bmatrix} \mathbf{w}_\ell[1, :]^\top \\ \vdots \\ \mathbf{w}_\ell[\mathcal{W}_\ell, :]^\top \end{bmatrix} = \begin{bmatrix} w_\ell[1, 0] & \dots & w_\ell[1, \mathcal{W}_{\ell-1}] \\ \vdots & & \vdots \\ w_\ell[\mathcal{W}_\ell, 0] & \dots & w_\ell[\mathcal{W}_\ell, \mathcal{W}_{\ell-1}] \end{bmatrix}$$

This matrix collects all learning parameters of layer ℓ

Forward Propagation: *Pseudo Code*

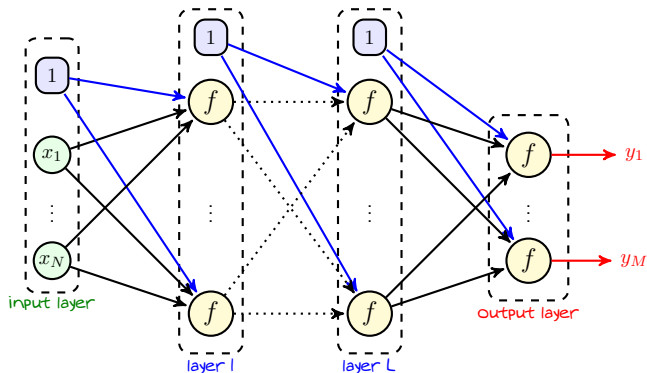
So, we can compactly present the forward propagation algorithm as follow

ForwardProp() :

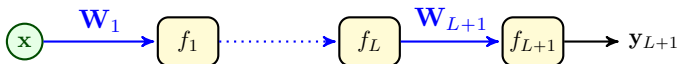
- 1: Initiate with $\mathbf{y}_0 = \mathbf{x}$
- 2: **for** $\ell = 0, \dots, L$ **do**
- 3: Add $\mathbf{y}_\ell[0] = 1$ and determine $\mathbf{z}_{\ell+1} = \mathbf{W}_{\ell+1}\mathbf{y}_\ell$ # forward affine
- 4: Determine $\mathbf{y}_{\ell+1} = f_{\ell+1}(\mathbf{z}_{\ell+1})$ # forward activation
- 5: **end for**
- 6: **for** $\ell = 1, \dots, L + 1$ **do**
- 7: Return $\mathbf{y}_\ell$ and $\mathbf{z}_\ell$
- 8: **end for**

Forward Propagation: *Compact Diagram*

Inspired by forward propagation, we can represent the FNN



by the following compact diagram



Fully-Connected FNNs: *Training*

For gradient descent: the main challenge is to compute $\nabla \hat{R}(\mathbf{w})$

We need partial derivatives of $\hat{R}(\mathbf{w})$ with respect to all w_ℓ

$$\begin{aligned}\frac{\partial}{\partial w_\ell [j, i]} \hat{R}(\mathbf{w}) &= \frac{\partial}{\partial w_\ell [j, i]} \frac{1}{B} \sum_{b=1}^B \mathcal{L}(\mathbf{y}_b, \mathbf{v}_b) \\ &= \frac{1}{B} \sum_{b=1}^B \boxed{\frac{\partial}{\partial w_{i,j} [\ell]} \mathcal{L}(\mathbf{y}_b, \mathbf{v}_b)}\end{aligned}$$

Key Observation

*We need an algorithm that computes partial derivative for a **sample loss***

Fully-Connected FNNs: *Training*

Let's make an agreement: *we consider a data-point $\mathbf{x}$ with label $\mathbf{v}$ and write*

$$\frac{\partial}{\partial w_{\ell} [j, i]} \mathcal{L} (\text{PassF} (\mathbf{x} | \mathbf{w}), \mathbf{v}) = \frac{\partial}{\partial w_{\ell} [j, i]} \mathcal{L} (\mathbf{y}, \mathbf{v})$$

while keeping in mind that $\mathbf{y}$ is a function of $\mathbf{w}$

To determine the partial derivatives, we note that

$\mathbf{y}$ is a nested function of $\mathbf{w}$

so, we can determine the derivative via *chain rule*. Let's recall the *chain rule* and see how we can *apply it on a graph*

Review: Chain Rule

Assume $z = g(x)$ and $y = f(z)$: y is a *nested* function of x , as we can write

$$y = f(g(x))$$

Intuitively, we can say: if at point x we move with tiny step dx , z varies as

$$dz = \dot{g}(x)dx$$

This variation also varies y : moving from $z = g(x)$ with tiny step dz leads to

$$dy = \dot{f}(z)dz$$

So, we have

$$dy = \dot{f}(z)\dot{g}(x)dx$$

Review: *Chain Rule*

We have concluded that by moving x with dx , we get

$$dy = \dot{f}(z)\dot{g}(x)dx$$

On the other hand, we know that

$$dy = \frac{d}{dx}f(g(x))dx$$

This concludes the *chain rule*

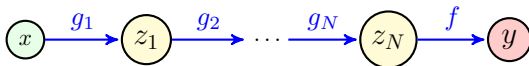
Chain Rule: *Scalar Form*

The derivative of *nested* function $y = f(g(x))$ with respect to x is given by

$$\frac{dy}{dx} = \frac{d}{dx}f(g(x)) = \dot{f}(z)\dot{g}(x) = \frac{dy}{dz} \frac{dz}{dx}$$

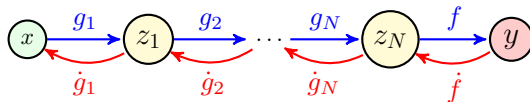
Computation Graph

We can represent the chain rule, using a *computation graph*: for the deep nested function given above, the computation graph is given by



In this graph, we start from x and pass forward to $z_1 \rightarrow z_2 \rightarrow \dots$ until we get to y . In each pass, we determine next variable via the function on the link

Computation Graph



$$\frac{dy}{dz_N} = \dot{f}(z_N)$$

$$\frac{dy}{dz_{N-1}} = \frac{dy}{dz_N} \frac{dz_N}{dz_{N-1}} = \dot{f}(z_N) \dot{g}_N(z_{N-1})$$

$$\vdots$$

$$\frac{dy}{dz_1} = \frac{dy}{dz_N} \frac{dz_N}{dz_{N-1}} \dots \frac{dz_2}{dz_1} = \dot{f}(z_N) \dot{g}_N(z_{N-1}) \dots \dot{g}_2(z_2)$$

$$\frac{dy}{dx} = \frac{dy}{dz_N} \frac{dz_N}{dz_{N-1}} \dots \frac{dz_2}{dz_1} \frac{dz_1}{dx} = \dot{f}(z_N) \dot{g}_N(z_{N-1}) \dots \dot{g}_2(z_2) \dot{g}_1(x)$$

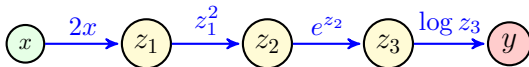
Computation Graph: *Example*

Example: y is a nested function of x through the following chain of functions:

$$z_1 = 2x \quad z_2 = z_1^2 \quad z_3 = e^{z_2} \quad y = \log z_3$$

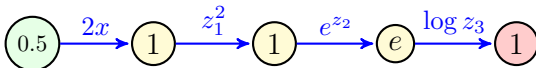
Determine the derivative of y with respect to x at $x = 0.5$.

Let's first plot the computation graph



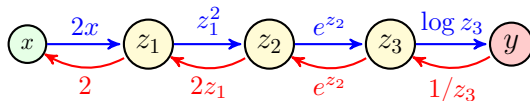
During the *forward pass* we get

$$z_1 = 2 \times 0.5 = 1 \rightarrow z_2 = 1^2 = 1 \rightarrow z_3 = e^1 = e \rightarrow y = \log e = 1$$

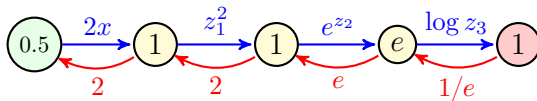


Computation Graph: *Example*

Now, we *pass backward* to determine the derivative

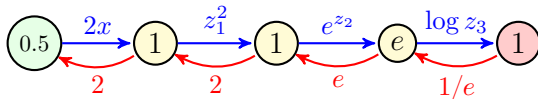


Let's first enter the values into the backward links



We now *navigate backward* to each variable that we want to determine the derivative y with respect to it

Computation Graph: *Example*



The derivatives are easily determined *recursively*

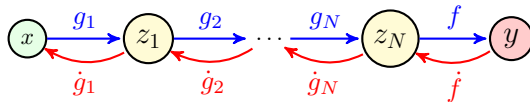
$$\frac{dy}{dz_3} = \frac{1}{e}$$

$$\frac{dy}{dz_2} = \frac{dy}{dz_3} \frac{dz_3}{dz_2} = \frac{e}{e} = 1$$

$$\frac{dy}{dz_1} = \frac{dy}{dz_2} \frac{dz_2}{dz_1} = 1 \times 2 = 2$$

$$\frac{dy}{dx} = \frac{dy}{dz_1} \frac{dz_1}{dx} = 2 \times 2 = 4$$

Computation Graph



In the example, we had to *first compute the value of each variable*, in order to be able to compute the *values of the backward links*

This is an important fact that we should remember

Backward pass is only possible if we have already taken the forward pass

Review: Chain Rule

The nested function can be a multivariate: *assume for* $n = 1, \dots, N$

$$z_n = g_n(x)$$

and let the nested function be

$$y = f(z_1, \dots, z_N)$$

Let's follow the same logic: *starting from point* x , *we move with tiny step* dx

$$dz_n = \dot{g}_n(x) dx$$

These variations lead to variation dy *in the nested function*

$$dy = \nabla f(\mathbf{z})^T d\mathbf{z} = \sum_{n=1}^N \frac{\partial y}{\partial z_n} dz_n = \sum_{n=1}^N \dot{f}_n(\mathbf{z}) dz_n$$

Review: Chain Rule

We can hence write

$$dy = \sum_{n=1}^N \dot{f}_n(\mathbf{z}) dz_n = \sum_{n=1}^N \dot{f}_n(\mathbf{z}) \dot{g}_n(x) dx$$

This concludes the *vector form* of the chain rule

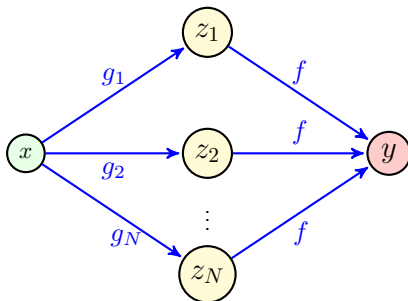
Chain Rule: Vector Form

Let $\mathbf{z} = [z_1, \dots, z_N]^T$ and $z_n = g_n(x)$. The derivative of nested function $y = f(\mathbf{z})$ with respect to x is given by

$$\frac{dy}{dx} = \sum_{n=1}^N \frac{\partial y}{\partial z_n} \frac{dz_n}{dx} = \sum_{n=1}^N \dot{f}_n(\mathbf{z}) \dot{g}_n(x)$$

Computation Graph

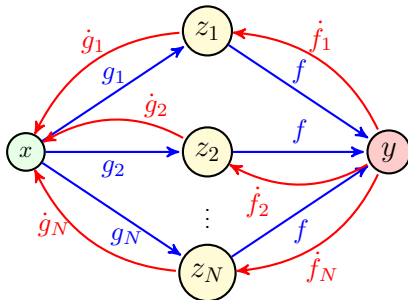
We can again represent the function via its vectorized *computation graph* and compute it by *passing forward* on this graph



We start from x and pass forward to $\mathbf{z} = [z_1, z_2, \dots, z_N]$. We then pass forward $\mathbf{z}$ to y . In each pass, we compute next variable via function on the link.

Computation Graph

The derivative with respect to any node is then given by *backward pass towards the node on the computation graph*

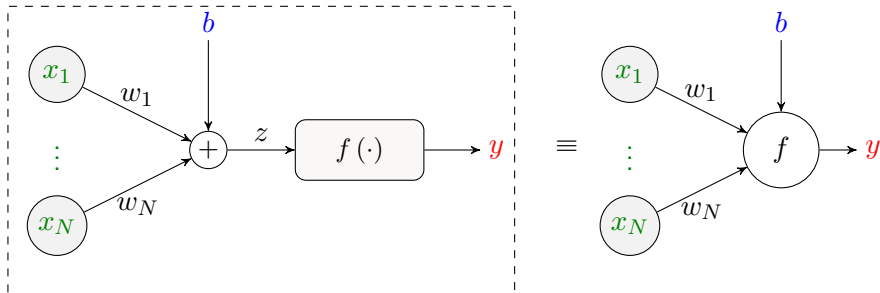


We add all backward passes towards x to determine the derivative

$$\frac{dy}{dx} = \sum_{n=1}^N \frac{\partial y}{\partial z_n} \frac{dz_n}{dx} = \sum_{n=1}^N \dot{f}_n(\mathbf{z}) \dot{g}_n(x)$$

Computation Graph: *Single Neuron*

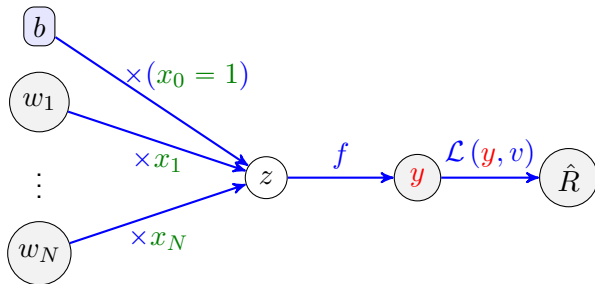
Let's now plot the *computation graph* of a single neuron and determine the gradient of the loss by *backward pass*



After passing the data-point $\mathbf{x}$ through the neuron, we get y and we calculate the loss for the *true label* v as $\mathcal{L}(y, v)$

Computation Graph: *Single Neuron*

The *computation graph* is hence given by

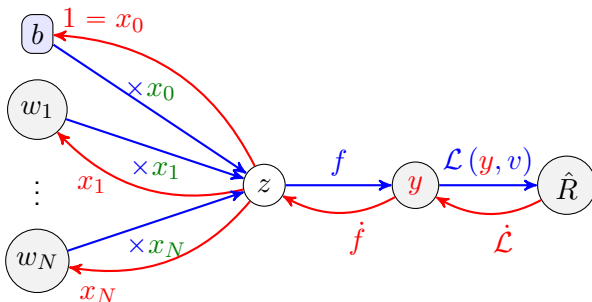


Here, the computation nodes are the *weights and bias of the neuron*

once we fix them, we can pass forward and get to the loss $\hat{R}$

Computation Graph: *Single Neuron*

Once passed forward, we can move backward to determine the derivatives

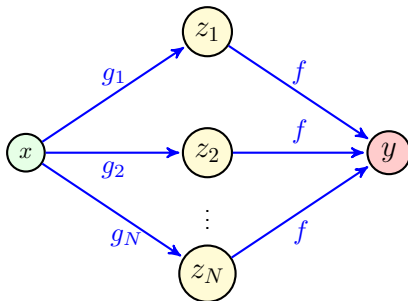


For a particular weight w_n , we can write (we drop arguments whenever clear)

$$\frac{\partial \hat{R}}{\partial w_n} = \frac{d\hat{R}}{dy} \frac{dy}{dz} \frac{\partial z}{\partial w_n} = \dot{\mathcal{L}} \dot{f} x_n$$

Computation Graph: *Multivariate Form*

Let's get back to the following *computation graph*



We define *vector-valued functions*, and show the graph *compactly*: let's define

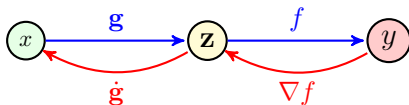
$$\mathbf{g}(x) = \begin{bmatrix} g_1(x) \\ \vdots \\ g_N(x) \end{bmatrix}$$

Computation Graph: *Multivariate Form*

We could define the derivative $\dot{\mathbf{g}}(\cdot)$ as the vector of derivatives $\dot{g}(\cdot)$

$$\dot{\mathbf{g}}(x) = \begin{bmatrix} \dot{g}_1(x) \\ \vdots \\ \dot{g}_N(x) \end{bmatrix} = \begin{bmatrix} \frac{dz_1}{dx} \\ \vdots \\ \frac{dz_N}{dx} \end{bmatrix} = \frac{d\mathbf{z}}{dx}$$

Let's show this *vectorized derivative* and *gradient* of f on the backward links



Well, we can *pass backward* as follows

$$\frac{dy}{dx} = \sum_{n=1}^N \frac{\partial y}{\partial z_n} \frac{dz_n}{dx} = \nabla f(\mathbf{z})^\top \dot{\mathbf{g}}(x)$$

Single Neuron: *Multivariate Form*

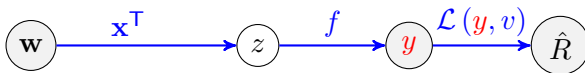
Let's define, as we did earlier, the following vectors

$$\mathbf{x} = \begin{bmatrix} x_0 = 1 \\ x_1 \\ \vdots \\ x_N \end{bmatrix} \quad \text{and} \quad \mathbf{w} = \begin{bmatrix} w_0 = b \\ w_1 \\ \vdots \\ w_N \end{bmatrix}$$

Recall that *output* of the *neuron* is determined as $y = f(z)$ for

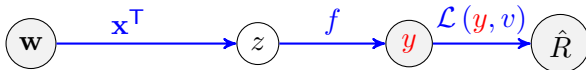
$$z = \mathbf{x}^T \mathbf{w}$$

So, we can show the *computation graph* compactly as



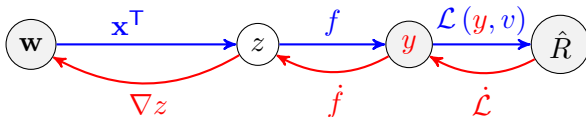
Computation Graph: *Multivariate Form*

Let's look at each link carefully: we pass *backward*, so we start with *last* link

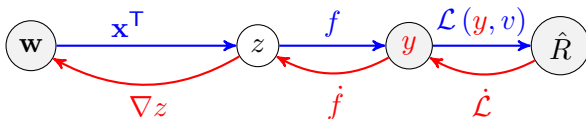


- $\hat{R}$ is a scalar function of scalar y , i.e., $\hat{R} = \mathcal{L}(y, v)$
 ↳ the backward link contains the *scalar derivative* $\dot{\mathcal{L}}$
- y is a scalar function of scalar z , i.e., $y = f(z)$
 ↳ the backward link contains the *scalar derivative* $\dot{f}$
- z is a scalar function of vector $\mathbf{w}$, i.e., $z = \mathbf{x}^\top \mathbf{w}$
 ↳ the backward link contains the *gradient* ∇z

So, the graph with the *backward* links looks like



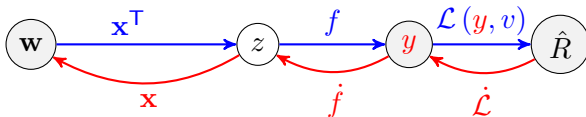
Computation Graph: *Multivariate Form*



We are almost complete; only we need to *calculate* ∇z

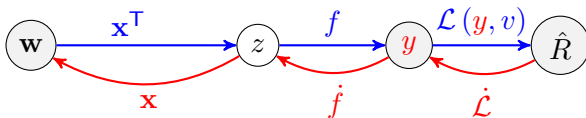
$$z = w_0 + w_1 x_1 + \dots + w_N x_N \rightsquigarrow \nabla z = \begin{bmatrix} \partial z / \partial w_0 \\ \partial z / \partial w_1 \\ \vdots \\ \partial z / \partial w_N \end{bmatrix} = \begin{bmatrix} 1 = x_0 \\ x_1 \\ \vdots \\ x_N \end{bmatrix} = \mathbf{x}$$

So, we are complete! Here is the *vectorized computation graph* of the *neuron*



Computation Graph: *Multivariate Form*

Now, how do we pass *backward* on this graph?



We arrived at y at the end of forward pass: at this point, we can determine

$$\frac{d\hat{R}}{dy} = \dot{\mathcal{L}}(y, v) = \dot{\mathcal{L}}$$

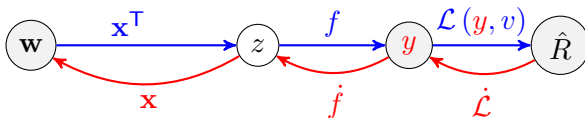
and we are at the *computing node* y . We then pass *backward* $\dot{\mathcal{L}}$. At node z , we can compute $\dot{f}(z)$, and use what we received from y to compute

$$\frac{d\hat{R}}{dz} = \frac{d\mathcal{L}}{dy} \frac{dy}{dz} = \dot{f} \dot{\mathcal{L}}$$

and pass it *backward*

Computation Graph: *Multivariate Form*

Now, how do we pass *backward* on this graph?



Arriving at w , we can determine $\nabla z = \dot{x}$ and use what we received from z to compute *what we want*

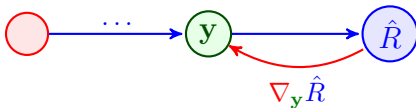
$$\nabla \hat{R} \text{ w.r.t. } \mathbf{w} \equiv \nabla_{\mathbf{w}} \hat{R} = \frac{d\hat{R}}{dz} \nabla z = \dot{f} \dot{\mathcal{L}} \dot{x}$$

Backpropagation: *Local Operations*

Let's consider a general problem: an *objective scalar* $\hat{R}$ is a function of *K -dimensional vector* $\mathbf{y} \in \mathbb{R}^K$. Clearly in this case, we have a *gradient*

$$\nabla_{\mathbf{y}} \hat{R} = \begin{bmatrix} \partial \hat{R} / \partial y_1 \\ \vdots \\ \partial \hat{R} / \partial y_K \end{bmatrix}$$

Assume that we know this gradient. The *vector* $\mathbf{y}$ is also function of *another variable*. We want to compute *gradient of $\hat{R}$* with respect to this *other variable*

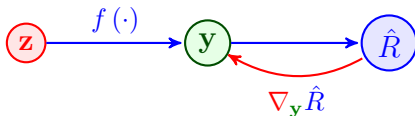


We now consider different cases for the *other variable* and its link to $\mathbf{y}$

Backpropagation: *Local Operation 1*

Entry-wise Functional Operation

$\mathbf{y} \in \mathbb{R}^K$ is a function of $\mathbf{z} \in \mathbb{R}^K$ as $\mathbf{y} = f(\mathbf{z})$ with $f(\cdot)$ operating entry-wise



For this case, we note that y_k is *only a function of* z_k ; thus we have

$$\frac{\partial \hat{R}}{\partial z_k} = \frac{\partial \hat{R}}{\partial y_k} \frac{\partial y_k}{\partial z_k} = \frac{\partial \hat{R}}{\partial y_k} \dot{f}(z_k)$$

So, we can use *entry-wise product* $\odot$ to get from $\nabla_{\mathbf{y}} \hat{R}$ to $\nabla_{\mathbf{z}} \hat{R}$

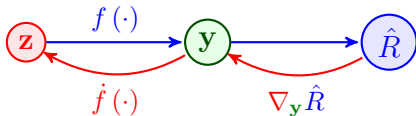
$$\nabla_{\mathbf{z}} \hat{R} = \nabla_{\mathbf{y}} \hat{R} \odot \dot{f}(\mathbf{z})$$

Backpropagation: *Local Operation 1*

Reminder: Entry-wise product of two vectors of the same size is

$$\mathbf{z} \odot \mathbf{y} = \begin{bmatrix} z_1 \\ \vdots \\ z_K \end{bmatrix} \odot \begin{bmatrix} y_1 \\ \vdots \\ y_K \end{bmatrix} = \begin{bmatrix} y_1 z_1 \\ \vdots \\ y_K z_K \end{bmatrix}$$

So, we can compactly perform this local operation as follows



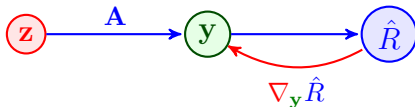
with the backward step

$$\nabla_{\mathbf{z}} \hat{R} = \nabla_{\mathbf{y}} \hat{R} \odot \dot{f}(\mathbf{z})$$

Backpropagation: *Local Operation 2*

Linear Vector-to-Vector Operation

$\mathbf{y} \in \mathbb{R}^K$ is a function of $\mathbf{z} \in \mathbb{R}^N$ as $\mathbf{y} = \mathbf{A}\mathbf{z}$ with $\mathbf{A} \in \mathbb{R}^{K \times N}$



Here, y_k is a linear function of $z_1, \dots, z_N$

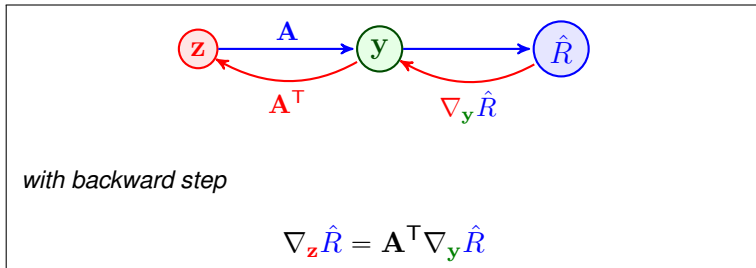
$$y_k = \sum_{n=1}^N \mathbf{A}[k, n] z_n$$

where $\mathbf{A}[k, n]$ is entry of $\mathbf{A}$ at row k and column n . We thus can write

$$\frac{\partial \hat{R}}{\partial z_n} = \sum_{k=1}^K \frac{\partial \hat{R}}{\partial y_k} \frac{\partial y_k}{\partial z_n} = \sum_{k=1}^K \frac{\partial \hat{R}}{\partial y_k} \mathbf{A}[k, n]$$

Backpropagation: *Local Operation 2*

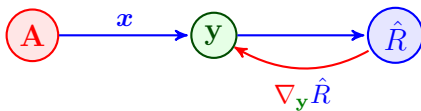
Long story short . . .



Backpropagation: *Local Operation 3*

Linear Matrix-to-Vector Operation

$\mathbf{y} \in \mathbb{R}^K$ is a function of $\mathbf{A} \in \mathbb{R}^{K \times N}$ as $\mathbf{y} = \mathbf{A}\mathbf{x}$ with $\mathbf{x} \in \mathbb{R}^N$



- + Wait a moment! The *other variable* is a matrix! How do we define $\nabla_{\mathbf{A}} \hat{R}$?
- Right! Let's first extend the definition

Backpropagation: *Local Operation 3*

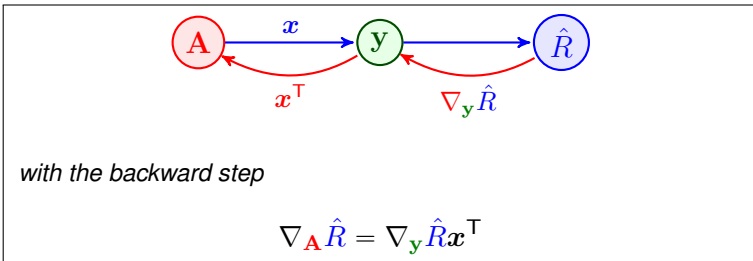
Assume scalar $\hat{R}$ is a function of matrix $\mathbf{A} \in \mathbb{R}^{K \times N}$, we define

$$\nabla_{\mathbf{A}} \hat{R} = \begin{bmatrix} \partial \hat{R} / \partial \mathbf{A} [1, 1] & \dots & \partial \hat{R} / \partial \mathbf{A} [1, N] \\ \vdots & & \vdots \\ \partial \hat{R} / \partial \mathbf{A} [K, 1] & \dots & \partial \hat{R} / \partial \mathbf{A} [K, N] \end{bmatrix}$$

with $\mathbf{A} [k, n]$ being the entry of $\mathbf{A}$ at row k and column n

Backpropagation: *Local Operation 3*

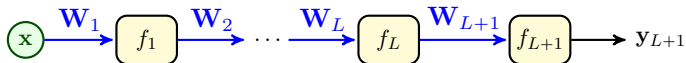
Long story short . . .



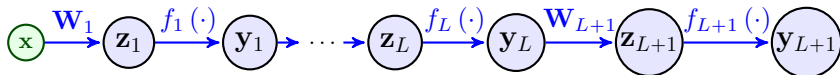
We are can also “*backpropagate*” over an MLP!

Backpropagation: *Algorithm*

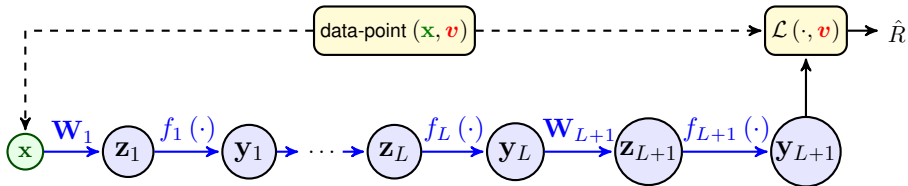
Let's recall the compact diagram of an FNN with L hidden layers



We can easily expand it into a *computation graph*

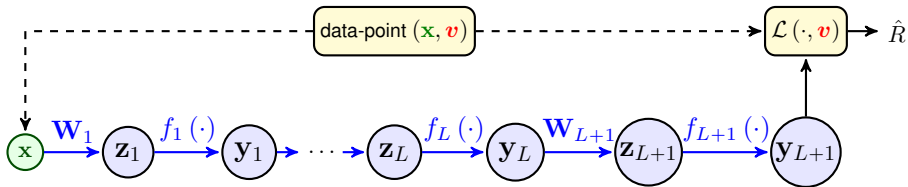


Our objective is *the empirical risk*; so let's include it also in the graph



Backpropagation: *Algorithm*

Given data-point $\mathbf{x}$ and its true label $\mathbf{v}$, we once complete a forward pass



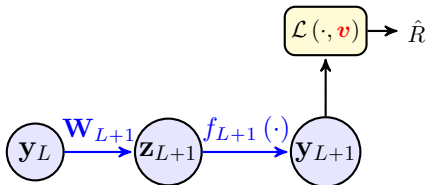
At the end of forward pass,

we know the value of all variables, i.e., $\mathbf{z}_\ell$ and $\mathbf{y}_\ell$ for all ℓ

Now, let's assume we want to find $\nabla_{\mathbf{W}_{L+1}} \hat{R}$

Backpropagation: *Algorithm*

We now cut the graph at the link $\mathbf{W}_{L+1}$



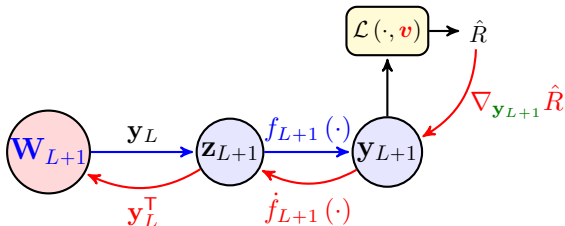
Let's recall . . .

- + *what is the variable here?*
- It's $\mathbf{W}_{L+1}$
- + *Can we modify the graph such that it becomes a node?*
- Sure! We note that $\mathbf{z}_{L+1} = \mathbf{W}_{L+1}\mathbf{y}_L$. We can look at it as a linear *matrix-to-vector* operation; so, we could modify the graph as

Backpropagation: *Algorithm*

Let's now move backward to $\mathbf{W}_{L+1}$

1. We have $\mathbf{y}_{L+1}$, so we compute $\nabla_{\mathbf{y}_{L+1}} \hat{R}$, and pass it
2. We have $\mathbf{z}_{L+1}$, so we compute $\dot{f}_{L+1}(\mathbf{z}_{L+1})$, and then we get $\nabla_{\mathbf{z}_{L+1}} \hat{R}$
3. We have $\mathbf{y}_L$, so we compute $\nabla_{\mathbf{W}_{L+1}} \hat{R}$ from the last pass as $\nabla_{\mathbf{W}_{L+1}} \hat{R}$



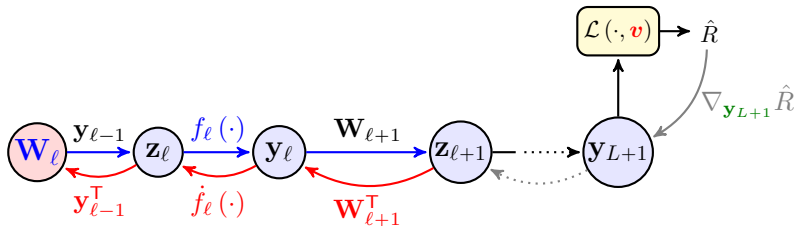
Backpropagation: *Algorithm*

We can *propagate* backward deeper and deeper

- We cut at the link that *we want to compute the gradient with respect to*
- We exchange the linear vector-to-vector function *at that particular link* to a linear *matrix*-to-vector function
- We move backwards till we get to the source of this graph

Backpropagation: *Algorithm*

As we arrive backward at layer ℓ , we already have messages till $\mathbf{z}_{\ell+1}$



Once we propagate back to the input, i.e., $\ell = 1$, then we have all gradients!

Backpropagation: *Few Notations*

To formally present backpropagation, *let us define a few notations*

For $\ell = 1, \dots, L + 1$, we define

$$\overleftarrow{\mathbf{y}}_{\ell} = \nabla_{\mathbf{y}_{\ell}} \hat{R}$$

$$\overleftarrow{\mathbf{z}}_{\ell} = \nabla_{\mathbf{z}_{\ell}} \hat{R}$$

and keep in mind that

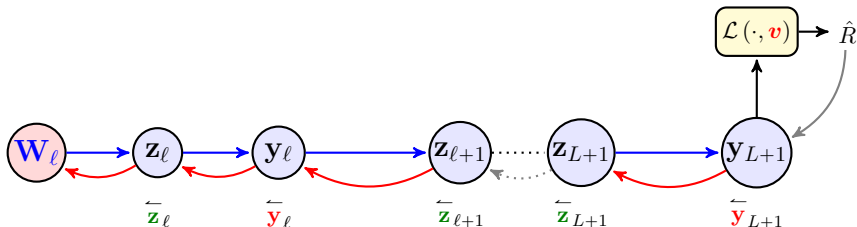
- $\mathbf{y}_{\ell}$ and $\overleftarrow{\mathbf{y}}_{\ell}$ are totally different things
- $\mathbf{z}_{\ell}$ and $\overleftarrow{\mathbf{z}}_{\ell}$ are totally different things

Backpropagation: *Pseudo Code*

```

1: Initiate with  $\bar{\mathbf{y}}_{L+1} = \nabla \mathcal{L}(\mathbf{y}_{L+1}, \mathbf{v})$  and  $\bar{\mathbf{z}}_{L+1} = \bar{\mathbf{y}}_{L+1} \odot \dot{f}_{L+1}(\mathbf{z}_{L+1})$ 
2: for  $\ell = L, \dots, 1$  do
3:   Determine  $\bar{\mathbf{y}}_{\ell} = \mathbf{W}_{\ell+1}^T \bar{\mathbf{z}}_{\ell+1}$  and drop  $\bar{\mathbf{y}}_{\ell}[0]$  # backward affine
4:   Determine  $\bar{\mathbf{z}}_{\ell} = \dot{f}_{\ell}(\mathbf{z}_{\ell}) \odot \bar{\mathbf{y}}_{\ell}$  # backward activation
5: end for
6: for  $\ell = 1, \dots, L + 1$  do
7:   Return  $\nabla_{\mathbf{w}_{\ell}} \hat{R} = \bar{\mathbf{z}}_{\ell} \mathbf{y}_{\ell-1}^T$ 
8: end for

```



Backpropagation: *Pseudo Code*

```
1: Initiate with  $\bar{\mathbf{y}}_{L+1} = \nabla \mathcal{L}(\mathbf{y}_{L+1}, \mathbf{v})$  and  $\bar{\mathbf{z}}_{L+1} = \bar{\mathbf{y}}_{L+1} \odot \dot{f}_{L+1}(\mathbf{z}_{L+1})$ 
2: for  $\ell = L, \dots, 1$  do
3:   Determine  $\bar{\mathbf{y}}_{\ell} = \mathbf{W}_{\ell+1}^T \bar{\mathbf{z}}_{\ell+1}$  and drop  $\bar{\mathbf{y}}_{\ell}[0]$  # backward affine
4:   Determine  $\bar{\mathbf{z}}_{\ell} = \dot{f}_{\ell}(\mathbf{z}_{\ell}) \odot \bar{\mathbf{y}}_{\ell}$  # backward activation
5: end for
6: for  $\ell = 1, \dots, L + 1$  do
7:   Return  $\nabla \mathbf{w}_{\ell} \hat{R} = \bar{\mathbf{z}}_{\ell} \mathbf{y}_{\ell-1}^T$ 
8: end for
```

+ *This looks very similar to forward propagation! Right?!*

– Yeah! Just we go *backward*! That's the whole point of backpropagation

You need to go once forth and then back to determine all gradients

Backpropagation: *Pseudo Code*

ForwardProp():

```

1: Initiate with  $\mathbf{y}_0 = \mathbf{x}$ 
2: for  $\ell = 0, \dots, L$  do
3:   Add  $\mathbf{y}_\ell[0] = 1$  and determine  $\mathbf{z}_{\ell+1} = \mathbf{W}_{\ell+1}\mathbf{y}_\ell$            # forward affine
4:   Determine  $\mathbf{y}_{\ell+1} = f_{\ell+1}(\mathbf{z}_{\ell+1})$            # forward activation
5: end for
6: for  $\ell = 1, \dots, L + 1$  do
7:   Return  $\mathbf{y}_\ell$  and  $\mathbf{z}_\ell$ 
8: end for

```

BackProp():

```

1: Initiate with  $\hat{\mathbf{y}}_{L+1} = \nabla \mathcal{L}(\mathbf{y}_{L+1}, \mathbf{v})$  and  $\hat{\mathbf{z}}_{L+1} = \hat{\mathbf{y}}_{L+1} \odot \dot{f}_{L+1}(\mathbf{z}_{L+1})$ 
2: for  $\ell = L, \dots, 1$  do
3:   Determine  $\hat{\mathbf{y}}_\ell = \mathbf{W}_{\ell+1}^\top \hat{\mathbf{z}}_{\ell+1}$  and drop  $\hat{\mathbf{y}}_\ell[0]$            # backward affine
4:   Determine  $\hat{\mathbf{z}}_\ell = \dot{f}_\ell(\mathbf{z}_\ell) \odot \hat{\mathbf{y}}_\ell$            # backward activation
5: end for
6: for  $\ell = 1, \dots, L + 1$  do
7:   Return  $\nabla \mathbf{w}_\ell \hat{R} = \hat{\mathbf{z}}_\ell \mathbf{y}_{\ell-1}^\top$ 
8: end for

```

Complete Training Loop via *Gradient Descent*

- + Say we use backpropagation; then, how does *gradient descent* look?
- Well! We should go back and forth with all data-points

Say we have dataset

$$\mathbb{D} = \{(\mathbf{x}_b, \mathbf{v}_b) \text{ for } b = 1, \dots, B\}$$

GradientDescent():

```
1: Initiate with some initial values  $\{\mathbf{W}_\ell^{(0)}\}$  and set a learning rate  $\eta$ 
2: while weights not converged do
3:   for  $b = 1, \dots, B$  do
4:     NN.values  $\leftarrow$  ForwardProp  $(\mathbf{x}_b, \{\mathbf{W}_\ell^{(t)}\})$  # forward
5:      $\{\nabla_{\mathbf{W}_\ell^{(t)}} \hat{R}_b\} \leftarrow$  BackProp  $(\mathbf{x}_b, \mathbf{v}_b, \{\mathbf{W}_\ell^{(t)}\}, \text{NN.values})$  # backward
6:   end for
7:   for  $\ell = 1, \dots, L + 1$  do
8:      $\mathbf{W}_\ell^{(t+1)} \leftarrow \mathbf{W}_\ell^{(t)} - \eta \text{ mean}(\nabla_{\mathbf{W}_\ell^{(t)}} \hat{R}_1, \dots, \nabla_{\mathbf{W}_\ell^{(t)}} \hat{R}_B)$ 
9:   end for
10: end while
```



UNIVERSITY OF
TORONTO

Further Read

Further Read

Read More on These Topics



Goodfellow, et al., *Deep Learning*

↳ [Chapters 5 and 6](#)



Bishop & Bishop, *Deep Learning: Foundations and Concepts*

↳ [Chapter 8](#)

Nice to Check

1. *Learning representations by back-propagating errors*

↳ D. Rumelhart et al. (1986)